

UNIT-I – Introduction to Data Structure and its Characteristics

Data, Data Structure and its Types

1 What is Data?

Data is a collection of raw facts, figures, symbols, or observations that can be processed by a computer to produce **meaningful information**.



Examples of Data:

- Names
- Images
- Numbers
- Text
- Marks
- Dates

2 What is Data Structure?

A data structure is a way of **organizing and storing** data in a computer so that it can be **accessed and used efficiently**. It represents the logical or mathematical organization of data and its implementation in a computer program.



Main Categories of Data Structures:

- 1 Linear Data Structures – Array, Linked List, Stack, Queue
- 2 Non-Linear Data Structures – Tree, Graph

3 Linear Data Structures

In linear data structures, data elements are placed in a sequential or linear order.

Types of Linear Data Structures

Primitive (Basic) Data Structures

- Integer
- Float
- Character
- Boolean

Non-Primitive (Derived) Data Structures

- Array
- Linked List
- Stack
- Queue

4 Examples of Linear Data Structures

Array



A collection of elements stored in contiguous memory locations.

Linked List



Elements are stored in non-contiguous memory locations.

Stack



Follows LIFO (Last In First Out).

Queue



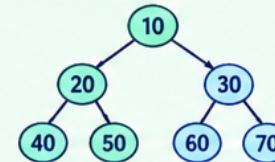
Follows LIFO (First In First Out).

5 Non-Linear Data Structures

Data structures where data elements are **not placed sequentially or linearly** are called **non-linear data structures**.

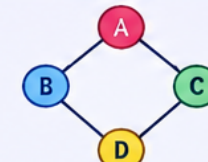
Types of Non-Linear Data Structures

Tree



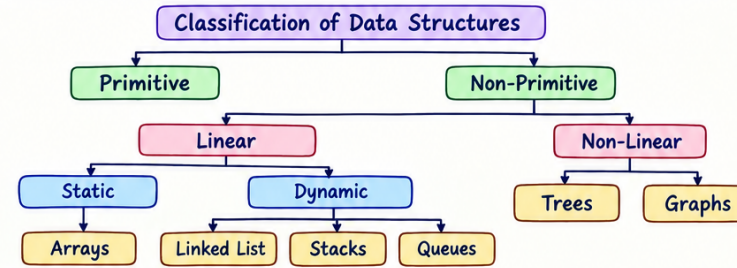
Hierarchical structure with one-to-many relationship.

Graph



Collection of nodes and edges with many-to-many relationship.

1.4



1.1 Primitive Data Structures

Definition: "It is the fundamental data structure and they have operations predefined by the compilers. Their representation are different on different computer systems."

Following are some examples of fundamental data types:

- 1) **int:** This is used for storing the integer value.
- 2) **char:** This is used for storing a single character.
- 3) **float and double:** They are used for storing decimal numbers.

1.1.2. Non-Primitive Data Structures

Definition: "Non-Primitive data structures are generated from primitive data structures. They can be structured as a collection of homogeneous or heterogeneous data items."

Non-Primitive data structures can be categorised into two types:

1) Linear Data Structures:

Definition: Linear data structure is a single level structure where the elements create a linear sequence. The elements of linear data structures are processed in sequential manner. The examples of linear data structures are stack, queue, array, linked list. Linear Data Structure can be categorised into two types;

- i) **Static Data Structures:** In static data structure, the size of the structure is fixed, i.e., the number of elements cannot be changed. An example of this type is an array.
- ii) **Dynamic Data Structures:** In this data structure, the size of data structure is variable, i.e., the number of elements can be increased or decreased in the structure dynamically. New elements can be added in the structure and existing elements can be removed from the structure at runtime. Following are examples of linear dynamic data structure:
 - Linked List
 - Stack
 - Queues

2) Non-Linear Data Structures:

Definition: These are multilevel data structures where elements are connected to multiple other elements in the structure so the processing is possible in non-linear way. In this data structure, the deletion and insertion of elements in linear way is not possible. Following are some examples of non-linear data structure:

- i) Trees
- ii) Graphs

☆ Quick Revision

Type	Examples
Primitive	int, char, float, double
Linear	Array, Linked List, Stack, Queue
Static	Array
Dynamic	Linked List, Stack, Queue
Non-Linear	Trees, Graphs

Arrays

Introduction

An array means collection. In an array, similar types of data are stored and these data are stored in the contiguous locations. It is also known as 'composite data structure' because data are organised sequentially. It is also known as homogeneous data structure because similar types of data are saved.

An array is a homogeneous, fixed-size collection of elements of the same data type which are allocated contiguously in memory.



Figure 1.4 : Concept of an Array

In the figure, the first address space saves the first element and the highest address saves the last one.

Characteristics of Arrays

- 1) Continuous memory locations are used to store the array element.
- 2) Lower bound cannot be changed after declaration, but with the help of compiler upper bound can be expanded.
- 3) In an array its name itself is a pointer.
- 4) Array elements can be differentiated by using element number while its name is same for all the elements.
- 5) Memory needs an array that depends on the number of elements and data types.
- 6) Total bytes = Size of (data type) × Size of array.
- 7) No memory locations can be inserted or removed if array is created but one can delete the elements.
- 8) An array has garbage values if after declaration it is not initialised.

Best, Worst, and Average-Case

- 1) **Worst-Case:** Maximum steps for input of size n .
(e.g., linear search: n comparisons)
 - 2) **Best-Case:** Minimum steps for input of size n .
(e.g., linear search: 1 comparison)
 - 3) **Average-Case:** Average number of steps for input of size n .
(e.g., linear search: $(n + 1)/2$ comparisons)
- All of the above cases can be defined by time function or size function.

Types of Arrays



Single-Dimensional Array

Representation of array elements in a single row is known as one dimensional array. It is also called linear array. It is represented as:

```
<data_type> array_name [size];
```

For example:

```
int marks[10];
```

Here, marks is the array name and 10 is the size.

Multi-Dimensional Array

Sometimes programmer needs to save the data in the tabular form. To store the data in this form C has the concept of two dimensional array (matrix).

Example:

```
int a[3][4];
```

This represents a 2D array with 3 rows and 4 columns.

Representation of Two-Dimensional Array

Consider the figure below which shows an array of $[m][n]$:

	0	1	2	...	...	n-1
0						
1						
2						
⋮						
m-1			a[i][k]			

Figure 1.8 : 2D Array representation

Here, i represents the row index and k represents the column index.

Declaration of One Dimensional Array

In C, an array is declared by specifying the data type, array name and size.

```
data_type array_name[size];
```

Example:

```
int nums[10];
```

Here, nums is the array name and 10 is the size.

Initialization of One Dimensional Array

Elements of an array can be initialised at the time of declaration or later.

Example:

```
int students[5] = {1,2,3,4,5};
```

Here, 5 elements are initialised.

➤ Row Major Order and Column Major Order ⚡

Example: 5 × 5 Array

Row Major Order

In row major order, the elements of a 2D array are stored row by row, i.e., all elements of the first row are stored first, then the second row, and so on.

5 × 5 Array (Row Major)

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

Storage order
(Row by row)

1, 2, 3, 4, 5,
6, 7, 8, 9, 10,
11, 12, 13, 14, 15,
16, 17, 18, 19, 20,
21, 22, 23, 24, 25

Address Formula (Row Major)

$$\text{Address} = \text{Base} + (i \times n + j) \times \text{Size}$$

Where,

i = row index j = column index

n = number of columns (here n = 5)

Example: Address of A[3][2] (Row Major)

$$\begin{aligned} \text{Address} &= \text{Base} + (3 \times 5 + 2) \times \text{Size} \\ &= \text{Base} + 17 \times \text{Size} \end{aligned}$$

Column Major Order

In column major order, the elements of a 2D array are stored column by column, i.e., all elements of the first column are stored first, then the second column, and so on.

5 × 5 Array (Column Major)

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

Storage order
(Column by column)

1, 6, 11, 16, 21,
2, 7, 12, 17, 22,
3, 8, 13, 18, 23,
4, 9, 14, 19, 24,
5, 10, 15, 20, 25

Address Formula (Column Major)

$$\text{Address} = \text{Base} + (j \times n + i) \times \text{Size}$$

Where,

i = row index j = column index

n = number of rows (here n = 5)

Example: Address of A[3][2] (Column Major)

$$\begin{aligned} \text{Address} &= \text{Base} + (2 \times 5 + 3) \times \text{Size} \\ &= \text{Base} + 13 \times \text{Size} \end{aligned}$$

Quick Comparison

Feature	Row Major Order	Column Major Order
Storage order	Row by row	Column by column
First accessed element	(0, 0), (0, 1), (0, 2), ...	(0, 0), (1, 0), (2, 0), ...
Commonly used in	C, C++, Java (2D arrays)	Fortran, MATLAB, scientific computing
Memory locality	Better for row-wise access	Better for column-wise access

— 5 × 5 Example | Row Major vs Column Major —

Sparse Matrices

Introduction

A sparse matrix is a matrix in which most of the elements are zero. Instead of storing all the elements, only the non-zero elements are stored to save memory and improve efficiency.

Key Terms

- Sparse Matrix** : Matrix with many zero elements.
- Dense Matrix** : Matrix with few or no zero elements.
- Square Matrix** : Number of rows = number of columns ($n \times n$).

Types of Sparse Matrices

1. Lower Triangular Matrix

All elements above the main diagonal are zero.

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & 3 & 0 & 0 \\ 4 & 5 & 6 & 0 \\ 7 & 8 & 9 & 10 \end{bmatrix}$$

Elements above the main diagonal are zero.

1D Storage (Lower Triangular)

1 2 3 4 5 6 7 8 9 10

Stored in row-wise order (only non-zero elements).
Total elements stored = 10 (for 4×4 matrix).

2. Upper Triangular Matrix

All elements below the main diagonal are zero.

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 0 & 5 & 6 & 7 \\ 0 & 0 & 8 & 9 \\ 0 & 0 & 0 & 10 \end{bmatrix}$$

Elements below the main diagonal are zero.

1D Storage (Upper Triangular)

1 2 3 4 5 6 7 8 9 10

Stored in row-wise order (only non-zero elements).
Total elements stored = 10 (for 4×4 matrix).

3. Tridiagonal Matrix

Non-zero elements occur only on the main diagonal and the two adjacent diagonals (upper and lower).

$$A = \begin{bmatrix} 1 & 2 & 0 & 0 & 0 \\ 3 & 4 & 5 & 0 & 0 \\ 0 & 6 & 7 & 8 & 0 \\ 0 & 0 & 9 & 10 & 11 \\ 0 & 0 & 0 & 12 & 13 \end{bmatrix}$$

Only three diagonals have non-zero elements.

1D Storage (Tridiagonal)

1 2 3 4 5 6 7 8 9 10 11 12 13

Stored in row-wise order (only non-zero elements).
Total elements stored = 13 (for 5×5 matrix).

General Formula

For a matrix $A[m][n]$ with non-zero elements stored in a 1D array:

- Lower Triangular:** $\text{index} = \frac{i(i+1)}{2} + j$ ($0 \leq j \leq i$)
- Upper Triangular:** $\text{index} = i \times n - \frac{i(i-1)}{2} + (j-i)$ ($i \leq j < n$)
- Tridiagonal:** $\text{index} = i \times n + j$ ($|i-j| \leq 1$)

Note: These formulas give the position of an element in the 1D array that stores only the non-zero elements.

Example: Sparse Matrix (5×5)

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 2 & 3 & 0 & 0 & 0 \\ 4 & 5 & 6 & 0 & 0 \\ 0 & 7 & 8 & 9 & 0 \\ 0 & 0 & 0 & 10 & 11 \end{bmatrix}$$

Non-zero elements
(Stored in 1D array):

1 2 3 4 5 6 7 8 9 10 11
(Total 11 elements)

Lower and Upper Triangular and Tridiagonal Matrix

Lower Triangular Matrix

Definition:

- A lower triangular matrix has all the elements above the main diagonal equal to zero.
- Only the elements on and below the main diagonal are stored.

Example:

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & 3 & 0 & 0 \\ 4 & 5 & 7 & 0 \\ 7 & 8 & 9 & 10 \end{bmatrix}$$

Elements above the main diagonal are zero.

1D Array Storage (Lower Triangular)

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

Stored in row-wise order (only non-zero elements).

Total elements stored = $\frac{n(n+1)}{2}$ (for $n = 4$, total = 10)

C Code Example:

```
#include <stdio.h>
int main(){
    int a[4][4]={1,0,0,0},{2,3,0,0},{4,5,6,0},{7,8,9,10};
    int v[10], i, j, k=0;
    for(i=0; i<4; i++){
        for(j=0; j<=i; j++){
            v[k++] = a[i][j];
        }
    }
    for(i=0; i<10; i++) printf("%d ", v[i]);
    return 0;
}
```

Key Points:

- Stored in row-wise order.
- Total elements = $n(n+1)/2$.
- Useful for sparse storage.

Upper Triangular Matrix

Definition:

- An upper triangular matrix has all the elements below the main diagonal equal to zero.
- Only the elements on and above the main diagonal are stored.

Example:

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 0 & 5 & 6 & 7 \\ 0 & 0 & 8 & 9 \\ 0 & 0 & 0 & 10 \end{bmatrix}$$

Elements below the main diagonal are zero.

1D Array Storage (Upper Triangular)

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

Stored in row-wise order (only non-zero elements).

Total elements stored = $\frac{n(n+1)}{2}$ (for $n = 4$, total = 10)

C Code Example:

```
#include <stdio.h>
int main(){
    int a[4][4]={1,2,3,4},{0,5,6,7},{0,0,8,9},{0,0,0,10};
    int v[10], i, j, k=0;
    for(i=0; i<4; i++){
        for(j=i; j<4; j++){
            v[k++] = a[i][j];
        }
    }
    for(i=0; i<10; i++) printf("%d ", v[i]);
    return 0;
}
```

Key Points:

- Stored in row-wise order.
- Total elements = $n(n+1)/2$.
- Useful for sparse storage.

Tridiagonal Matrix

Definition:

- A tridiagonal matrix has non-zero elements only on the main diagonal and the two adjacent diagonals (upper and lower).
- All other elements are zero.

Example:

$$A = \begin{bmatrix} 1 & 2 & 0 & 0 \\ 3 & 4 & 5 & 0 \\ 0 & 6 & 7 & 8 \\ 0 & 0 & 9 & 10 \end{bmatrix}$$

Only main diagonal, upper and lower diagonals have non-zero elements.

1D Array Storage (Tridiagonal)

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

Stored in row-wise order (only non-zero elements).

Total elements stored = $3n - 2$ (for $n = 4$, total = 10)

C Code Example:

```
#include <stdio.h>
#include <stdlib.h>
int main(){
    int a[4][4]={1,2,0,0},{3,4,5,0},{0,6,7,8},{0,0,9,10};
    int v[10], i, j, k=0;
    for(i=0; i<4; i++){
        for(j=0; j<4; j++){
            if(abs(i-j)<=1){
                v[k++] = a[i][j];
            }
        }
    }
    for(i=0; i<10; i++) printf("%d ", v[i]);
    return 0;
}
```

Key Points:

- Stored in row-wise order.
- Total elements = $3n - 2$.
- Useful for band matrices.

Vector and Vector Representation

1. Vector

Definition :

A vector is a one-dimensional array used to store a list of elements of the same data type. It is similar to a simple array, but is often used in programming to represent mathematical vectors (i.e., a sequence of numbers).

Key Points :

- 1D array of same data type elements.
- Elements are stored in contiguous memory locations.
- Can be accessed using an index (subscript).
- Size can be fixed or dynamic (depending on language).

2. Vector Representation

Definition :

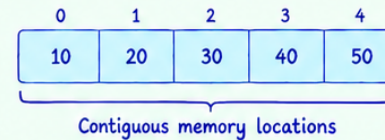
Vector can be represented in memory as a one-dimensional array. Each element has a position (index) from 0 to n-1.

Example :

Consider a vector V of 5 elements:

$V = [10, 20, 30, 40, 50]$

Memory representation:



3. Declaration and Initialization

Declaration :

To declare a vector in C, use the following syntax:

```
data_type array_name[size];
```

Example :

```
int v[5]; // vector of 5 integers
```

Initialization :

Values can be assigned at the time of declaration or later.

Example :

```
int v[5] = {10, 20, 30, 40, 50};  
// or  
int v[5];  
v[0] = 10; v[1] = 20; .. v[4] = 50;
```

4. Accessing Elements

Elements of a vector are accessed using the index (subscript) operator [].

Example :

```
int v[5] = {10, 20, 30, 40, 50};  
printf("%d", v[0]); // 10  
printf("%d", v[3]); // 40
```

Note: Index starts from 0 and goes up to size-1.

5. Vector Operations

1. Traversal :

Visit each element of the vector.

```
for (i = 0; i < n; i++)  
    printf("%d ", v[i]);
```

2. Search :

Find an element in the vector.

```
for (i = 0; i < n; i++)  
    if (v[i] == x) printf("Found");
```

3. Insertion :

Add a new element at a given position.

4. Deletion :

Remove an element from a given position.

6. Example Program (Vector)

Program to read and print a vector of 5 elements:

```
#include <stdio.h>  
int main() {  
    int v[5], i;  
    // Input  
    for(i = 0; i < 5; i++) {  
        printf("Enter element %d: ", i);  
        scanf("%d", &v[i]);  
    }  
    // Output  
    printf("Vector elements: ");  
    for(i = 0; i < 5; i++) {  
        printf("%d ", v[i]);  
    }  
    return 0;  
}
```

7. Vector vs Array (Quick Difference)

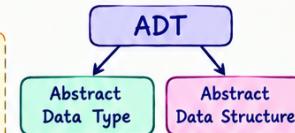
Feature	Vector	Array
Definition	1D array (often used to represent mathematical vector).	General term for an array (1D, 2D, etc.).
Dimension	Always 1D.	Can be 1D, 2D, multi-dimensional.
Usage	Used for simple list of elements, mathematical operations.	Used for storing data in different forms and structures.
Syntax (C)	data_type v[size];	data_type a[rows][cols]; (for 2D)

— Vector | 1D Array | Same Data Type —

1.1.3 Abstract Data Type (ADT)

1.1.3.1 Definition of Abstract Data Type (ADT)

An abstract data type is defined indirectly, only by the operations that may be performed on it and by mathematical constraints on the effects (and possibly cost) of those operations.



Both are mathematical models for similar behaviour or semantics.

1.1.3.2 How to understand the definition (in simple words)

- It does not describe how the data is stored.
- It only tells what operations can be performed (e.g., push, pop, insert, delete, etc.) and the rules/constraints on those operations (e.g., time, cost, or behaviour).

1.1.3.3 Key Points from the Definition

- An ADT is a mathematical model for a certain class of data structures.
- It is defined by two operations: Push, Pop (example).
- Each operation has a constraint and may return the most recently pushed item.
- It abstracts the implementation details of the data structure.

1.1.3.1 Types of ADT

1 Value Definition

Value definition is further divided into two parts:

- Definition Clause:** As the names suggests the definition clause states the concepts of the data types. Definition clause is mandatory.
- Condition Clause:** Condition clause defines any condition that applies to the data type. Condition clause is optional.

2 Operation Definition

There are three parts of operational definition:

- Function:** The function clause defines the role of the operation. If we consider the addition operation in integers, the function clause will state that two integers can be added using this function.
- Pre-Condition:** This clause specifies any restrictions that must be satisfied before the operation can be applied. This clause is optional. If we consider the division operation on integers then the pre-condition will state that the divisor should not be zero.
- Post-Condition:** Post condition specifies what the operation does that is, the result of the operation is performed.

1.1.3.2 Examples of ADT

- Vector:** Direct access to each element and fixed size.
- List:** Unbounded. Fast access to first element and thereafter sequential access.
- Stack:** LIFO structure. Insertions and deletions from one end only.
- Queue:** FIFO structure. Insertions and deletions from the ends.
- Tree:** Fast insertions and access. Hierarchical structures.
- Hash Table:** Fast access.
- Dictionary:** Values associated with key.
- Graph:** Collection of nodes and arcs. Unordered.
- File:** External storage of persistent data. Slow access.
- Set:** Fast unions and intersections, Elements not maintained in order.
- Bag:** As with sets but allows multiple occurrences of elements.

— Design by Sarafraj Navaz —

Analysis of Algorithm

Correctness of Algorithm

Definition: An algorithm is correct if, for every input instance, it halts with the correct output.

- It must produce the desired output for all legal inputs.
- It must not run forever (must halt).
- It must always give correct output.

Complexity of Algorithms

Definition: It measures the efficiency of an algorithm in terms of time and memory required.

- Helps compare algorithms.
- Two main resources: Time and Memory.
- Often expressed using a framework called Analysis Framework.

Types of Complexity

Space Complexity

Definition: The amount of memory space required by an algorithm to store data.

Includes:

- Space for input
- Space for output
- Space for variables and temporary data

Measured in: Number of bytes (or units of memory).

Time Complexity

Definition: The amount of time taken by an algorithm to complete its execution.

Depends on:

- Number of basic operations
- Input size

Measured in: Number of steps (or operations).

Complexity Types (Summary)

Space Complexity

- Memory space required.
- Includes input, output, variables.
- Measured in bytes/units.

Time Complexity

- Time taken to execute.
- Depends on input size.
- Measured in number of steps.

Time Complexity of Some Basic Operations

Operation	Cost (units)	Repetition No. of times	Total Time (worst case)
Line (sumOfList)	1	1	1
for (i = 0; i < n; i++)	1 + 1 + 1	1 + (n + 1) + n	2n + 2
sum = sum + A[i]	2	n	2n
return sum	1	1	1
Total Time Required			4n + 4

Best, Worst, and Average-Case

- **Worst-Case:** Maximum steps for input of size n. (e.g., linear search: n comparisons)
- **Best-Case:** Minimum steps for input of size n. (e.g., linear search: 1 comparison)
- **Average-Case:** Average number of steps for input of size n. (e.g., linear search: (n + 1)/2 comparisons)

All of the above cases can be defined by time function or size function.

Analysis Framework

Definition: Analysis of algorithms can be applied using a framework called Analysis Framework to evaluate its suitability for various applications and compare with other algorithms.

It helps to:

- Understand the behaviour of the algorithm.
- Find the time and space requirements.
- Compare and improve algorithms.

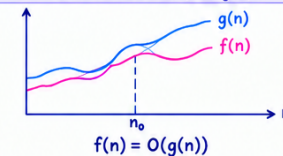
Asymptotic Notations

Definition: Notations represent the order of growth of a function. They show the behaviour of time and space complexities.

- 1) **Theta (Θ):** Tight bound (both upper and lower).
- 2) **Big-O (O):** Upper bound (worst case).
- 3) **Big-Omega (Ω):** Lower bound (best case).
- 4) **Little-o (o):** Strictly smaller bound.

It represents the shortest amount of execution time that an algorithm takes.

One can write $f(n) = O(g(n))$



— Design by Sarafraj Navaz —